Edges and Lines

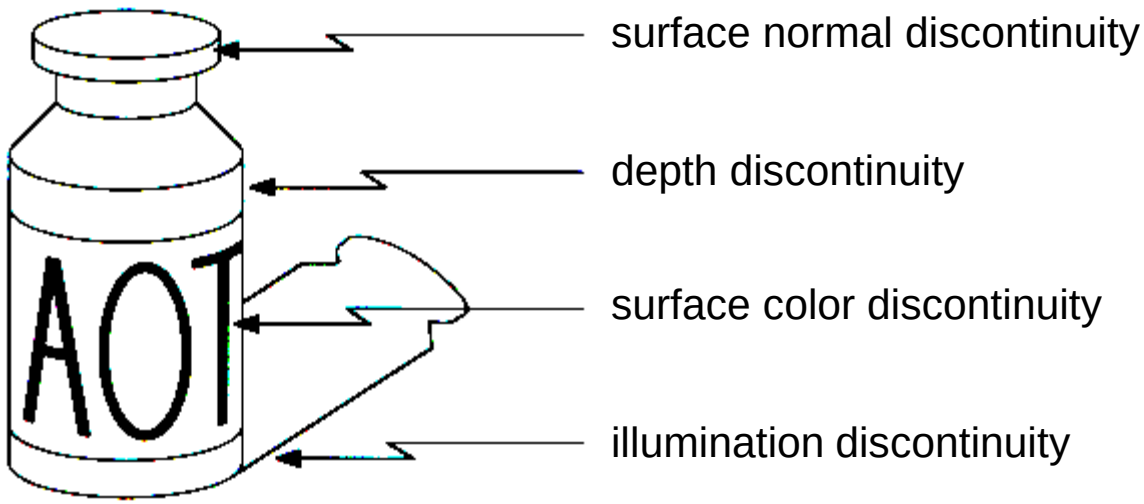
Detecting edges

What's an edge?

- intensity discontinuity (= rapid change)

How can we find large changes in intensity?

- gradient operator seems like the right solution

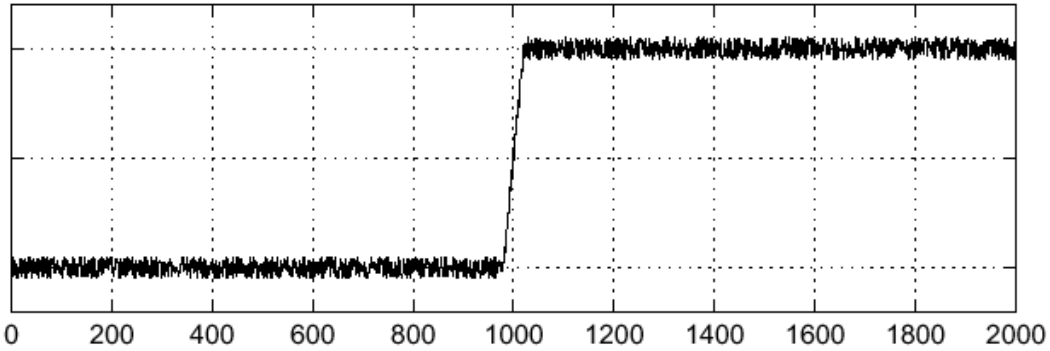


Effects of noise

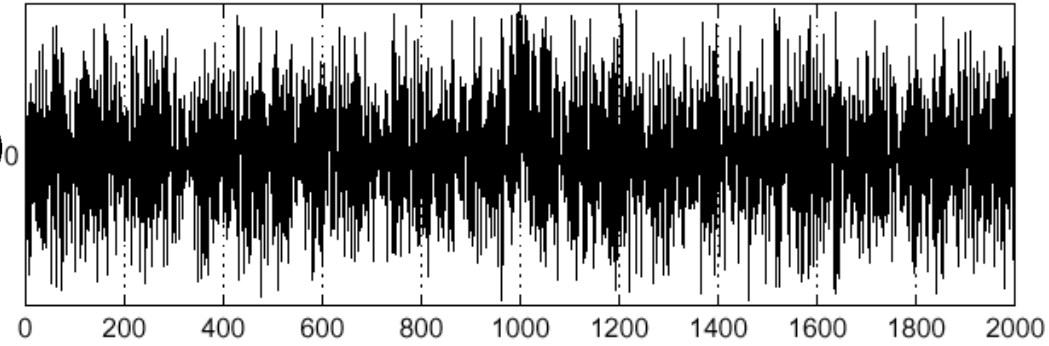
Consider a single row or column of the image

- Plotting intensity as a function of position gives a signal

$f(x)$



$\frac{d}{dx}f(x)$



Where is the edge?

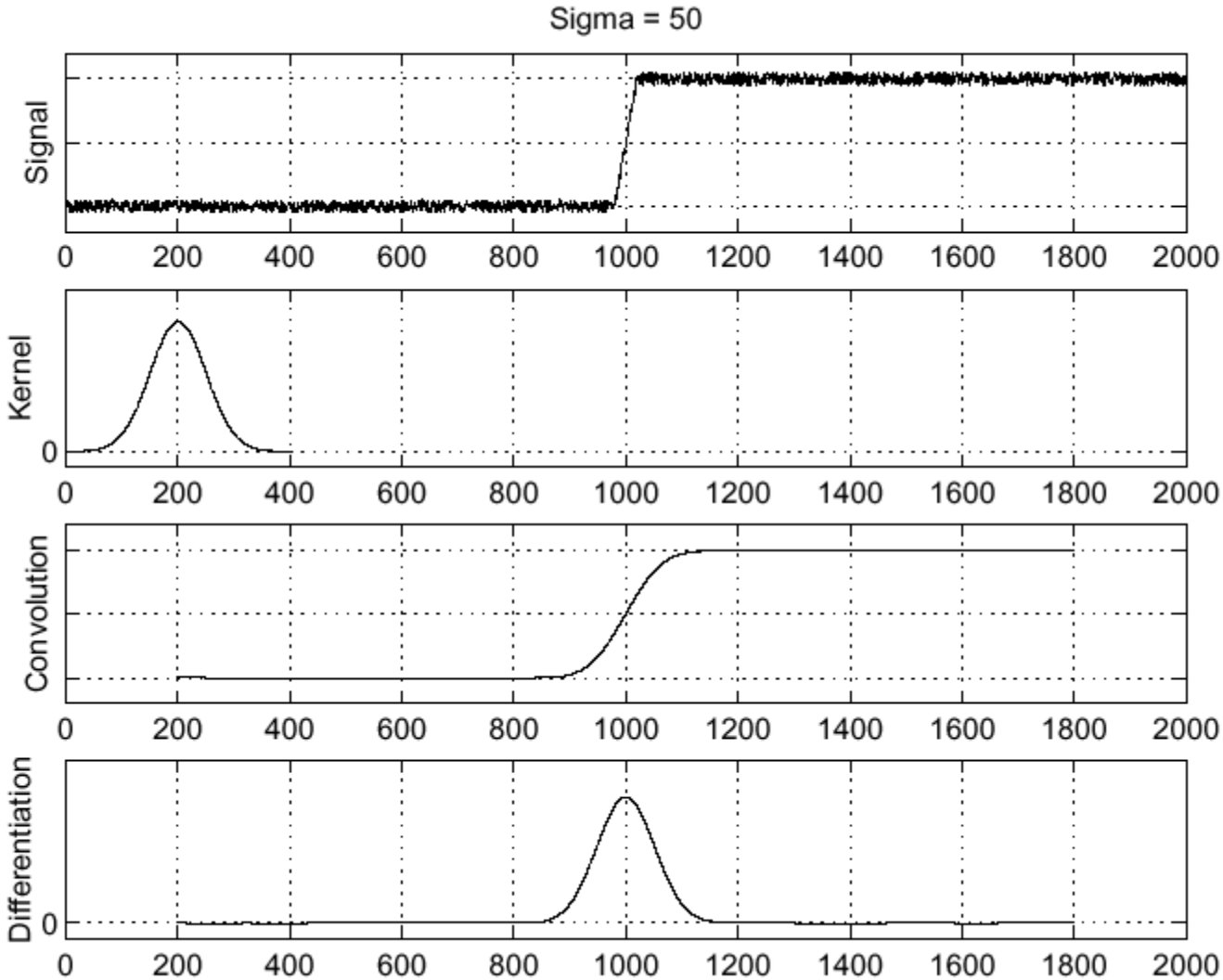
Solution: smooth first

f

h

$h \star f$

$\frac{\partial}{\partial x}(h \star f)$



Where is the edge? Look for peaks in $\frac{\partial}{\partial x}(h \star f)$

Associative property of convolution

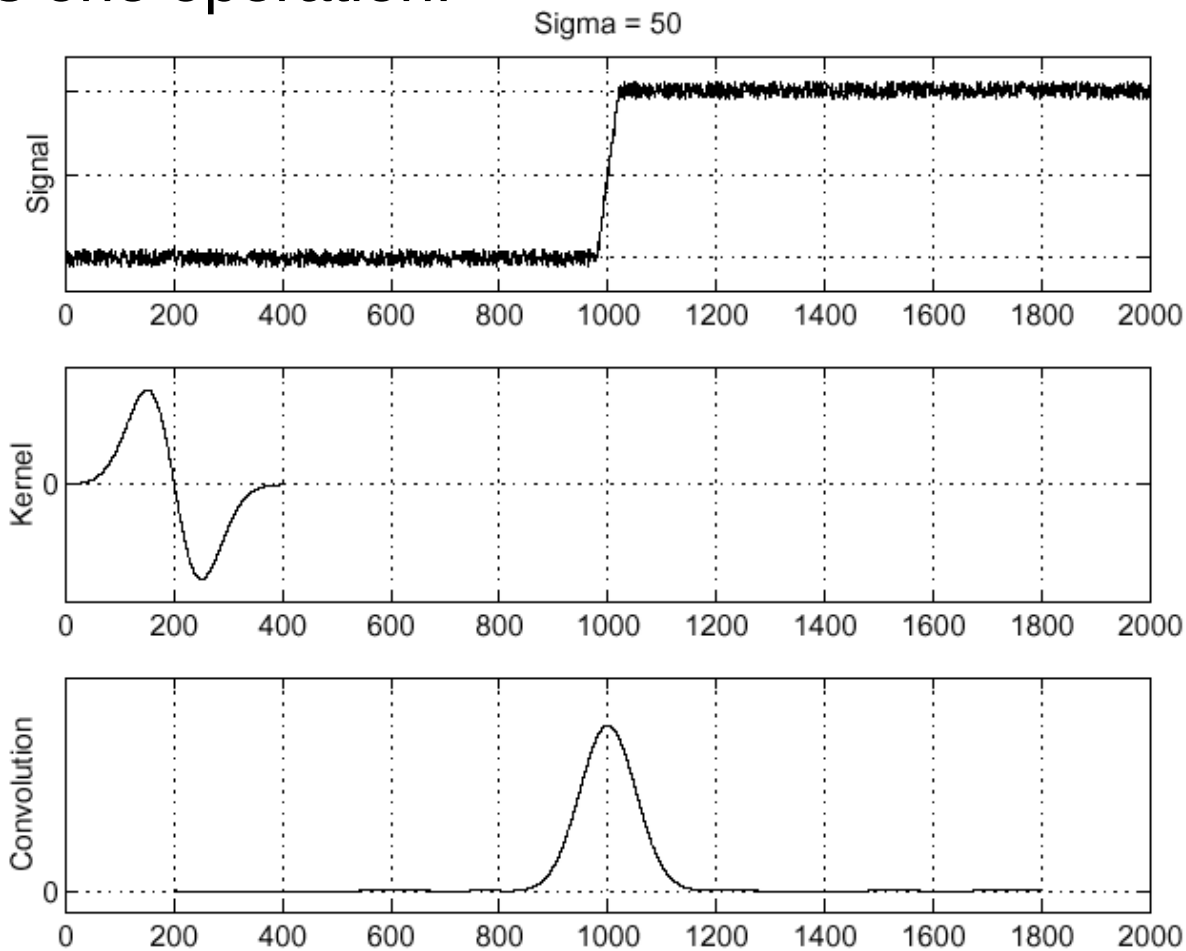
$$\frac{\partial}{\partial x}(h \star f) = (\frac{\partial}{\partial x}h) \star f$$

This saves us one operation:

f

$\frac{\partial}{\partial x}h$

$(\frac{\partial}{\partial x}h) \star f$



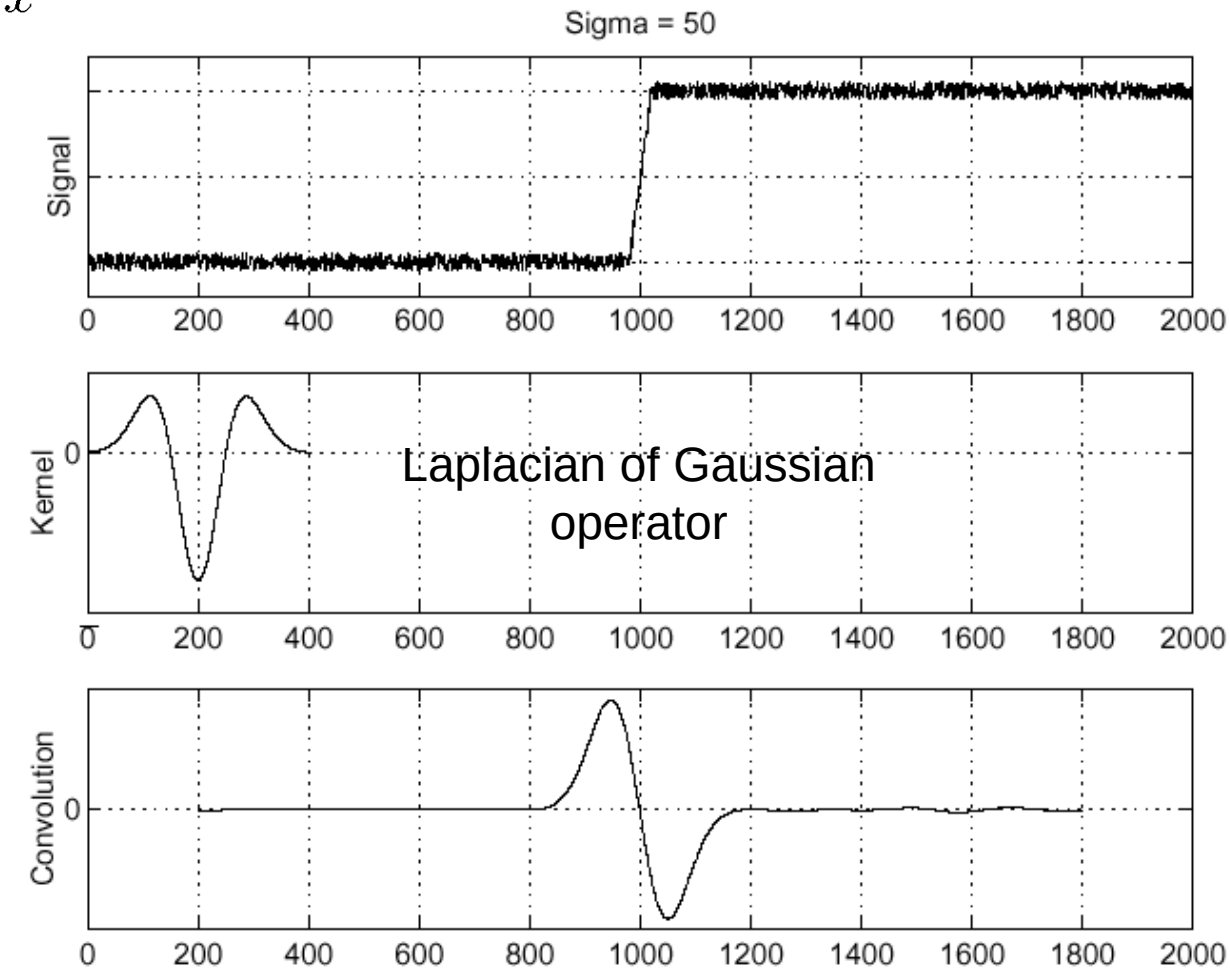
Laplacian of Gaussian

Consider $\frac{\partial^2}{\partial x^2}(h \star f)$

f

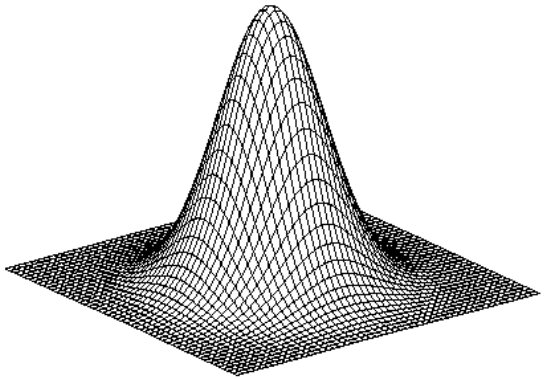
$\frac{\partial^2}{\partial x^2}h$

$(\frac{\partial^2}{\partial x^2}h) \star f$



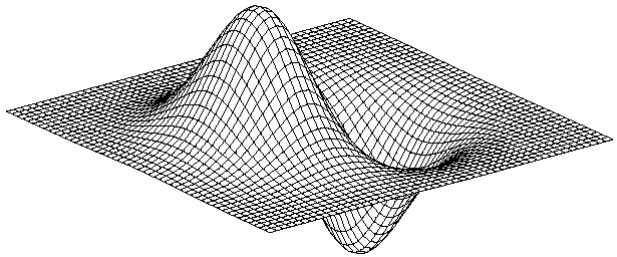
Where is the edge? Zero-crossings of bottom graph

2D edge detection filters



Gaussian

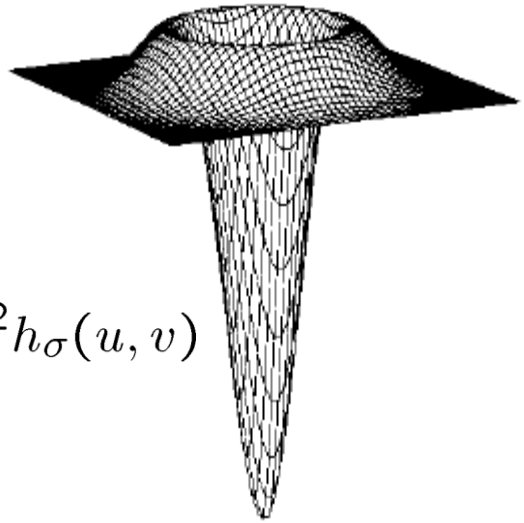
$$h_\sigma(u,v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



derivative of Gaussian

$$\frac{\partial}{\partial x} h_\sigma(u,v)$$

Laplacian of Gaussian



$$\nabla^2 h_\sigma(u,v)$$

∇^2 is the **Laplacian** operator:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Discrete Approximations

Common approximation of derivative of Gaussian

$$\frac{1}{8}\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \qquad \frac{1}{8}\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

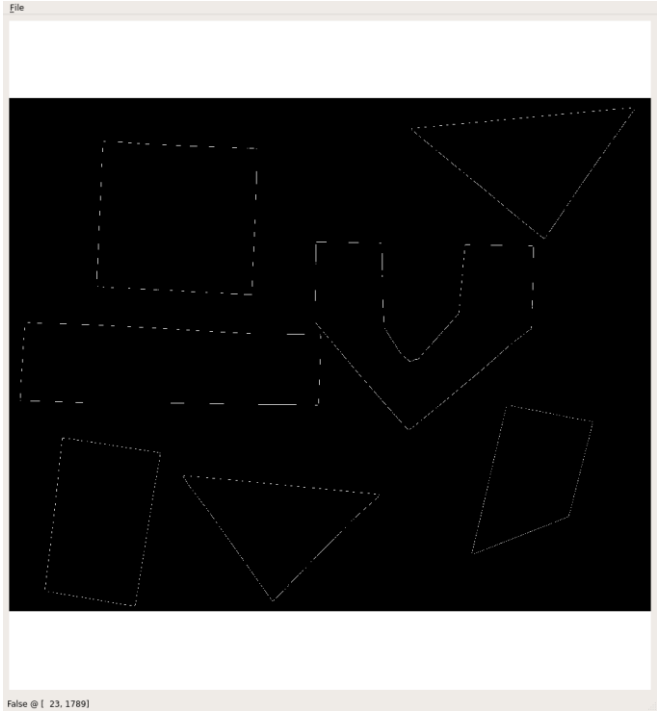
s_x $\qquad$ s_y

- The standard defn. of the Sobel operator omits the 1/8 term
  - doesn't make a difference for edge detection
  - the 1/8 term **is** needed to get the right gradient value, however

Laplacian Filter

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Edge Detection Results - Canny Edge Detection



Low Threshold + High Threshold

Fitting

Choose a parametric object/some objects to represent a set of tokens

Most interesting case is when criterion is not local

- can’t tell whether a set of points lies on a line by looking only at each point and the next.

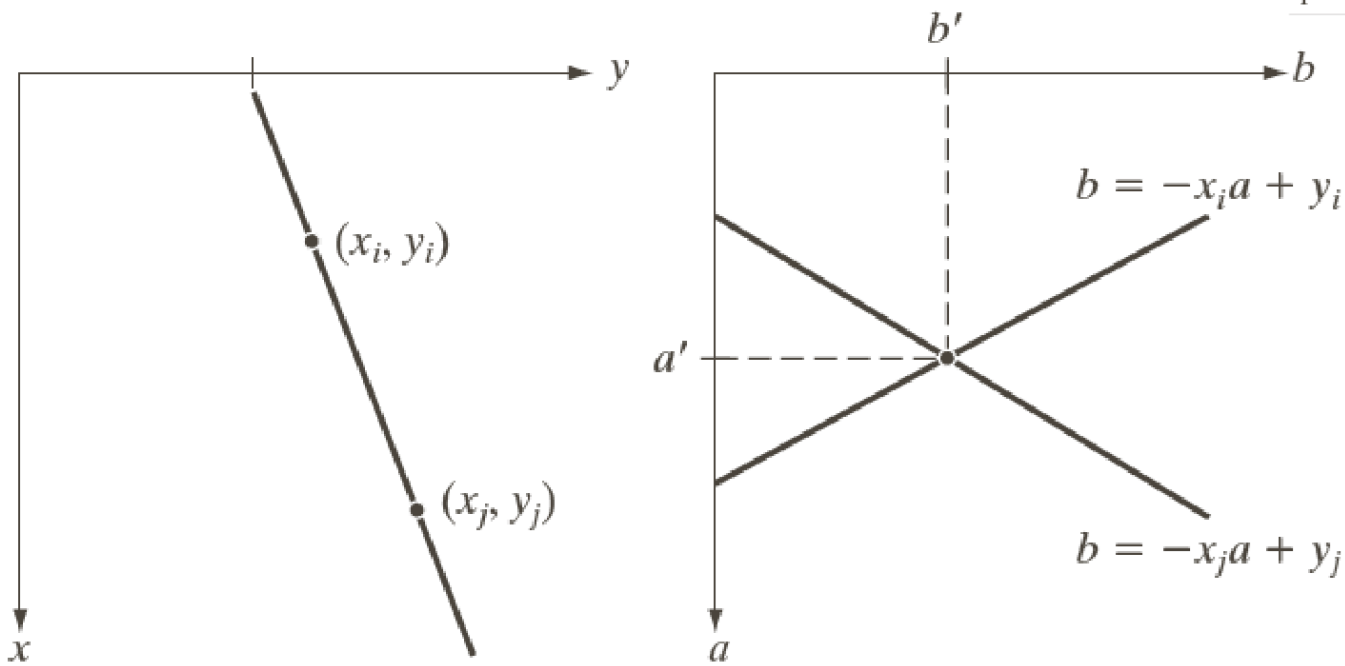
Three main questions:

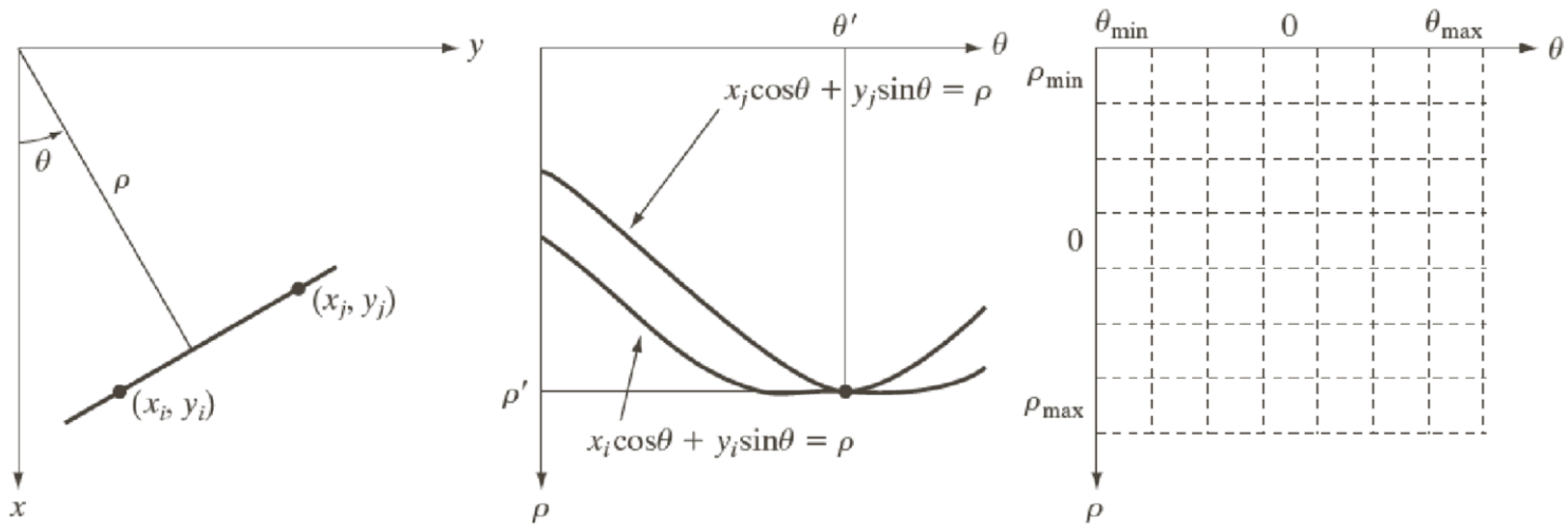
- what object represents this set of tokens best?
- which of several objects gets which token?
- how many objects are there?

(you could read line for object here, or circle, or ellipse or...)

a b

FIGURE 10.31 (a) xy-plane. (b) Parameter space.





a b c

FIGURE 10.32 (a) (ρ, θ) parameterization of line in the xy-plane. (b) Sinusoidal curves in the $\rho\theta$-plane; the point of intersection (ρ', θ') corresponds to the line passing through points (x_i, y_i) and (x_j, y_j) in the xy-plane. (c) Division of the $\rho\theta$-plane into accumulator cells.

Mechanics of the Hough transform

Construct an array representing θ, d

For each point, render the curve (θ, d) into this array, adding one at each cell

Difficulties

- how big should the cells be? (too big, and we cannot distinguish between quite different lines; too small, and noise causes lines to be missed)

How many lines?

- count the peaks in the Hough array

Who belongs to which line?

- tag the votes

Hardly ever satisfactory in practice, because problems with noise and cell size defeat it

RANSAC

Choose a small subset uniformly at random

Fit to that

Anything that is close to result is signal; all others are noise

Refit

Do this many times and choose the best

Issues

- How many times?
  - Often enough that we are likely to have a good line
- How big a subset?
  - Smallest possible
- What does close mean?
  - Depends on the problem
- What is a good line?
  - One where the number of nearby points is so big it is unlikely to be all outliers

Algorithm RANSAC: fitting lines using random sample consensus

Determine:
- n — the smallest number of points required
- k — the number of iterations required
- t — the threshold used to identify a point that fits well
- d — the number of nearby points required to assert a model fits well

Until k iterations have occurred
- Draw a sample of n points from the data uniformly and at random
- Fit to that set of n points
- For each data point outside the sample
  - Test the distance from the point to the line against t; if the distance from the point to the line is less than t, the point is close
- end
- If there are d or more points close to the line then there is a good fit. Refit the line using all these points.

end

Use the best fit from this collection, using the fitting error as a criterion